

QUESTION 1

30m left

2. Tick Sizes

ALL

"Tick size is the minimum price change up or down of a trading instrument in a market." - Investopedia

You are given a set of tick size raw data from an exchange for a particular trading instrument.

Tick Size Raw Data from Exchange

Start Price	End Price	Minimum Price Increment
0	0.1	0.01
0.1	0.5	0.02
0.5	1	0.02
1	10	1
10	50	1
50		

The tick size raw data gives a price band for which a minimum price increment needs to be adhered to for the instrument. The price of an instrument cannot go up or down in smaller increments than its stated tick size.

For example, if an instrument last traded at \$0.30, the next price up is \$0.32 - you cannot sell it for \$0.31 as the tick size at that price band is \$0.02.

Another example is that if you would like to buy this instrument at a slightly higher price than \$1.00,

Test

30m left

Another example is that if you would like to buy this instrument at a slightly higher price than \$1.00, you can only submit a bid at \$2.00 as the tick size at that price band is \$1.00.

ALL

The last line in the tick size raw data stipulates the hypothetical maximum price of the trading instrument (i.e. you cannot ever trade above that price).

Task #1: Trim down the tick size rules to a more human-readable set of rules by switching to just a lower bound format.

This can be done by,

- Identifying and removing duplicate price increment intervals.
- Removing the upper bound of the tick size interval.

Tick Size Raw Data from Exchange

Start Price	End Price	Minimum Price Increment
0	0.1	0.01
0.1	0.5	0.02
0.5	1	0.02
1	10	1
10	50	1
50		

The example tick size raw data from an exchange will be translated into the following tick size rules.

Test Res

30m left

Tick Size Rules

ALL

Lower Price Bound	Tick Size
0	0.01
0.1	0.02
1	1
50	

Task #2: Use the tick size rules that you have generated to find the next possible incremental price of the trading instrument at specific prices.

Function Description
Complete the function `tick sizes` in the editor below.

`tick sizes` has the following parameter(s):
`raw_tick sizes List[double]`: raw-wise representation of the tick size raw data from exchange
`float prices List[double]`: specific prices of the trading instrument

Returns
`List[List[double], List[double]]`: row-wise representation of the human-readable set of rules and next incremental prices of the trading instrument

Assumptions

- Input `raw_tick sizes` will always be sorted in order of increasing start price.
- Input prices will always be at the correct tick size intervals (e.g. if the tick size interval is 1.0 between 1.0 and 5.0, input prices will only be 1.0, 2.0, 3.0, 4.0 and 5.0).
- Tick sizes will at most have 2 decimal places.

Notes

- Focus on delivering a working solution, this will be sufficient to pass the question.
- To achieve a perfect score, your solution must have optimal processing-time performance for large datasets.

Input Format for Custom Testing

Sample Case 0

Sample Input 0

```
0.0 0.1 0.01 0.1 0.5 0.02 0.5 1.0 0.02
1.0 10.0 1.0 10.0 50.0 1.0 50.0
0.3 1.0
```

Sample Output 0

```
0.0 0.01 0.1 0.02 1.0 1.0 50.0
0.32 2.0
```

Test Res

30m left

Returns

ALL

`List[List[double], List[double]]`: row-wise representation of the human-readable set of rules and next incremental prices of the trading instrument

Assumptions

- Input `raw_tick sizes` will always be sorted in order of increasing start price.
- Input prices will always be at the correct tick size intervals (e.g. if the tick size interval is 1.0 between 1.0 and 5.0, input prices will only be 1.0, 2.0, 3.0, 4.0 and 5.0).
- Tick sizes will at most have 2 decimal places.

Notes

- Focus on delivering a working solution, this will be sufficient to pass the question.
- To achieve a perfect score, your solution must have optimal processing-time performance for large datasets.

Input Format for Custom Testing

Sample Case 0

Sample Input 0

```
0.0 0.1 0.01 0.1 0.5 0.02 0.5 1.0 0.02
1.0 10.0 1.0 10.0 50.0 1.0 50.0
0.3 1.0
```

Sample Output 0

```
0.0 0.01 0.1 0.02 1.0 1.0 50.0
0.32 2.0
```

Test Result

#include <iostream>

#include <vector>

#include <algorithm>

#include <iomanip>

```
using namespace std;
```

```
struct TickSizeRule {  
    double lowerPriceBound;  
    double tickSize;  
};
```

```
void trimTickSizeRules(const vector<vector<double>> &raw_tickSizes, vector<TickSizeRule>  
&tickSizeRules) {
```

```
    double prevLower = raw_tickSizes[0][0];  
    double prevTickSize = raw_tickSizes[0][2];
```

```
    tickSizeRules.push_back({prevLower, prevTickSize});
```

```
    for (int i = 1; i < raw_tickSizes.size(); ++i) {  
        double currentLower = raw_tickSizes[i][0];  
        double currentTickSize = raw_tickSizes[i][2];
```

```
        if (currentTickSize != prevTickSize) {  
            tickSizeRules.push_back({currentLower, currentTickSize});  
            prevTickSize = currentTickSize;  
        }  
    }
```

```
}
```

```
double findNextIncrement(double price, const vector<TickSizeRule> &tickSizeRules) {
```

```
    auto it = lower_bound(tickSizeRules.begin(), tickSizeRules.end(), price, [](const TickSizeRule &rule,  
double value) {
```

```
        return rule.lowerPriceBound <= value;  
    });
```

```
    if (it != tickSizeRules.begin()) {  
        --it;  
    }
```

```
    return price + it->tickSize;  
}
```

```
int main() {
```

```
    vector<vector<double>> raw_tickSizes = {  
        {0.0, 0.2, 0.1},
```

```
        {0.2, 0.6, 0.2},
```

```
        {0.6, 1.0, 0.2},
```

```

    {1.0, 1.1, 0.1},

    {1.1, 1.6, 0.5},

    {1.6, 2.8, 0.3},

    {2.8}
};

vector<double> prices = {0.5, 1.3, 2};

vector<TickSizeRule> tickSizeRules;
trimTickSizeRules(raw_tickSizes, tickSizeRules);

for (const auto &rule : tickSizeRules) {
    cout << fixed << setprecision(2) << rule.lowerPriceBound << " " << rule.tickSize << endl;
}

for (double price : prices) {
    double nextPrice = findNextIncrement(price, tickSizeRules);
    cout << fixed << setprecision(2) << nextPrice << endl;
}

return 0;
}

```

QUESTION 2

<https://leetcode.com/problems/search-suggestions-system/description/?envType=problem-list-v2&envId=trie>

Input: vector of strings named words, string named search

Todo: Think of a search engine that has the words vector as its database. string search is your current keyword. You start typing the search word and your search engine starts giving 3 suggestions whose prefixes match the current state of your search word. So, as you write the search word character by character, you get a new recommendation list suited for that. If there are more than three words in the database that are suited, return the three lexicographically smallest ones. If there are none, return empty list.

Output: A vector<vector<string>> which contains the recommendations for all characters being typed one by one.

Example: words : ["cart", "carpet", "carpool", "call"], search: carl

output:

```

[
c : ["call", "carpet", "carpool"],
ca : ["call", "carpet", "carpool"],
car : ["carpet", "carpool", "cart"],
carl : []
]

```

]

```
#include <bits/stdc++.h>
#define ll long long
class Solution {
public:
    vector<vector<string>> suggestedProducts(vector<string>& products, string
searchWord) {
        sort(products.begin(), products.end());
        vector<vector<string>> result;
        string currentPrefix = "";
        for(auto&c : searchWord){
            currentPrefix+=c; // Build the prefix incrementally
            vector<string> suggestions;
            // Use lower_bound to find the first product that is not less than the
current prefix
            auto it = lower_bound(products.begin(), products.end(), currentPrefix);
            // Collect up to three valid suggestions
            int i = 0;
            while(i < 3 && it != products.end() && it->substr(0, currentPrefix.size())
== currentPrefix){
                suggestions.push_back(*it);
                ++i;
                ++it;
            }
            result.push_back(suggestions);
        }

        return result;
    }
};
```

QUESTION 3:

The screenshot shows a HackerRank problem page for '6. Debugging Exercise' with the subproblem 'Matching Market Participants'. The problem description involves matching buy and sell orders in a market. It includes two tables of orders and a list of trades.

Buy Orders

Buy Orders	Sell Orders
100 shares @ \$10.0/share	100 shares @ \$11.0/share
200 shares @ \$9.0/share	400 shares @ \$12.0/share

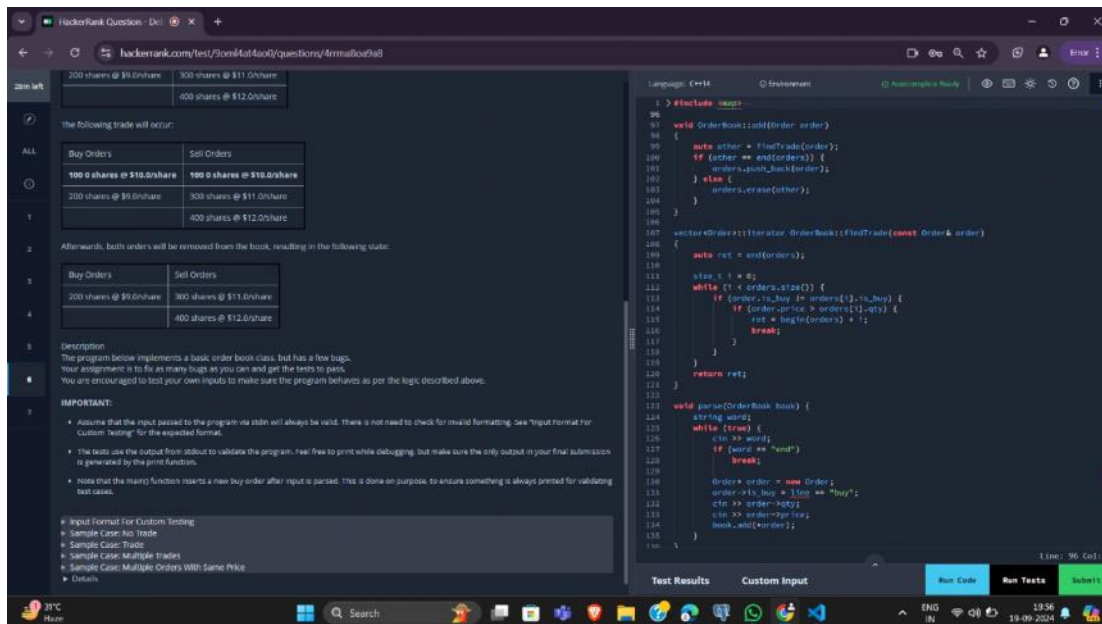
Sell Orders

Buy Orders	Sell Orders
100 shares @ \$10.0/share	100 shares @ \$10.0/share
200 shares @ \$9.0/share	300 shares @ \$11.0/share
	400 shares @ \$12.0/share

The following trade will occur:

Buy Orders	Sell Orders
100 0 shares @ \$10.0/share	100 0 shares @ \$10.0/share

The code editor on the right shows a C++ implementation of an order book. It includes functions for adding orders, finding the best bid/ask, and matching orders. The code is currently empty, with only the include and namespace declarations visible.



```
#include <map>
#include <set>
#include <list>
#include <cmath>
#include <ctime>
#include <deque>
#include <queue>
#include <stack>
#include <string>
#include <bitset>
#include <cstdio>
#include <limits>
#include <vector>
#include <climits>
#include <cstring>
#include <cstdlib>
#include <fstream>
#include <numeric>
#include <sstream>
#include <iomanip>
#include <iostream>
#include <algorithm>
#include <unordered_map>
using namespace std;
struct Order
{
    bool is_buy = false;
    int qty = 0;
    double price = 0.0;
};
ostream& operator <<(ostream& os, const Order& order)
{
    return os << (order.is_buy ? "buy " : "sell ")
        << order.qty << "@$" << setprecision(1)
        << fixed << order.price;
}
bool operator <(const Order& a, const Order& b)
{
    return a.price < b.price;
};
class OrderBook {
public:
    OrderBook() = default;

    // formats and prints the orders as the test cases expect.
```

```

~OrderBook() {
    auto itr = splitIntoBuyAndSellOrders();
    std::sort(begin(orders), itr);
    std::sort(itr, end(orders));
    print();
}
// checks the opposing side's available orders.
// for a buy order, look at existing sell orders, and vice versa.
// if a trade is possible, update the order book accordingly.
// otherwise, insert the order into the book.
void add(Order order);

// print the order book
// IMPORTANT: the test cases depend on this method.
// please make sure this is the ONLY output that gets printed to stdout.
void print();
private:
    // returns an iterator to the best "match" for a given order.
    // for buy orders, this would be the lowest sell price.
    // for sell orders, the highest buy price.
    // if no orders meet the criteria, return orders.end()
    vector<Order>::iterator findTrade(const Order& order);

    // split orders into buy and sell orders.
    // return an iterator to the first sell order.
    // all orders before the iterator are buy orders.
    // all orders after are sell orders.
    vector<Order>::iterator splitIntoBuyAndSellOrders();

    vector<Order> orders;
};
vector<Order>::iterator OrderBook::splitIntoBuyAndSellOrders() {
    return stable_partition(begin(orders),
                           end(orders),
                           [](const Order& order) { return order.is_buy; });
}
void OrderBook::print() {
    for (const auto& order : orders)
        cout << order << endl;
}

void OrderBook::add(Order order) {
    while (order.qty > 0) {
        auto other = findTrade(order);
        if (other == end(orders)) {
            // Aggregate orders if there is an existing order with the same price and type
            auto it = std::find_if(orders.begin(), orders.end(), [&order](const Order&
existingOrder) {
                return existingOrder.is_buy == order.is_buy && existingOrder.price ==
order.price;
            });
            if (it != orders.end()) {
                it->qty += order.qty;
            } else {
                orders.push_back(order);
            }
            return;
        } else {
            // Match found, fulfill as much as possible
            if (order.qty >= other->qty) {
                order.qty -= other->qty;
                orders.erase(other);
            } else {
                other->qty -= order.qty;
                order.qty = 0;
            }
        }
    }
}

```

```

    }
}
vector<Order>::iterator OrderBook::findTrade(const Order& order) {
    vector<Order>::iterator best_match = orders.end();
    for (auto it = orders.begin(); it != orders.end(); ++it) {
        if (order.is_buy != it->is_buy) {
            if (order.is_buy) {
                // Buy order: match with the lowest sell price
                if (it->price <= order.price) {
                    if (best_match == orders.end() || it->price < best_match->price) {
                        best_match = it;
                    }
                }
            } else {
                // Sell order: match with the highest buy price
                if (it->price >= order.price) {
                    if (best_match == orders.end() || it->price > best_match->price) {
                        best_match = it;
                    }
                }
            }
        }
    }
}
return best_match;
}
void parse(OrderBook &book) {
    string word;
    while (true) {
        cin >> word;
        if (word == "end")
            break;
        Order order;
        order.is_buy = (word == "buy");
        cin >> order.qty >> order.price;
        book.add(order);
    }
}
int main() {
    OrderBook book;
    parse(book);
    Order order = {true, 10, 11.0};
    book.add(order);
}

```

URL HASHING

1. URL Hashing

Implement an algorithm to hash a URL as described.

Suppose the given url url of length n is to be hashed with a string $hash_string$ of length m . Given an integer k , run the url through the following algorithm:

1. Divide the url into blocks of size k starting from the left. The last block can be smaller than k . For example, if $url = "https://xyz.com"$ and $k = 4$, the blocks are ["http", "s/", "xyz", ".com"].
2. The values of the English characters 'a', 'b', ..., 'z' are 0, 1, ..., 25 respectively and that of '!', '!' and ':' are 26, 27, and 28 respectively. Thus the hash value of the block 's/' will be $19 + 26 + 27 + 27 = 98$.
3. For each URL, find the hash value of each block. The hash value is the sum of the values of each character.
4. Replace the block with the (hash value of the block modulo m^{th} character of the string $hash_string$).

Given the string url , $hash_string$, and an integer k , find the hashed string.

Example
Suppose $url = "https://xyz.com"$, $hash_string = "pqrst"$, and $k = 4$.

Block	Hash Value	Hash Character
http	$7 + 19 + 19 + 15 = 60$	$hash_string[60 \% 5 = 0] = p$
s/	$18 + 26 + 27 + 27 = 98$	$hash_string[98 \% 5 = 3] = s$

Example
Suppose $url = "https://xyz.com"$, $hash_string = "pqrst"$, and $k = 4$.

Block	Hash Value	Hash Character
http	$7 + 19 + 19 + 15 = 60$	$hash_string[60 \% 5 = 0] = p$
s/	$18 + 26 + 27 + 27 = 98$	$hash_string[98 \% 5 = 3] = s$
xyz	$23 + 24 + 25 + 28 = 100$	$hash_string[100 \% 5 = 0] = p$
com	$2 + 14 + 12 = 28$	$hash_string[28 \% 5 = 3] = s$

Each hash value is divided by the length of $hash_string$ or 5 in this case. The remainders point to the characters in $hash_string$, and the answer is "psps".

Function Description
Complete the function `getHashedURL` in the editor below.

`getHashedURL` has the following parameters:

- `string url`: the input URL
- `string hash_string`: a hash string for mapping
- `int k`: the block sizes

Returns
`string`: the hash string of the URL

Constraints

```
#include <iostream>
```

```
#include <string>
```

```
#include <vector>
```

```
using namespace std;
```

```
int charValue(char ch) {
    if (ch >= 'a' && ch <= 'z')
        return ch - 'a';
    if (ch == '/')
        return 26;
    if (ch == '.')

```



```

        return 27;
    return -1;
}

string hashURL(const string& url, const string& hash_string, int k) {
    int n = url.size();
    int m = hash_string.size();
    string result = "";

    for (int i = 0; i < n; i += k) {
        int hash_value = 0;

        for (int j = i; j < i + k && j < n; j++) {
            hash_value += charValue(url[j]);
        }

        result += hash_string[hash_value % m];
    }

    return result;
}

int main() {
    string url = "https://xyz.com";
    string hash_string = "pqrst";
    int k = 4;

    string hashed_url = hashURL(url, hash_string, k);

    cout << "Hashed URL: " << hashed_url << endl;

    return 0;
}

```

COMPANY USERNAMES:

1h 5m left

2. Company Usernames

When the Company hires new employees, it needs to assign usernames to them. Your task is to implement a system to assign unique usernames to new hires at the Company.

Given an array of real-world names, *names*, return an array of usernames, *usernames*, each of which must be unique.

Username constraints:

1. Maximum length of the username is always 8 characters, and should be as close to 8 characters as possible
2. May only contains characters in the sets [a-z] and [0-9], beware real-world names containing hyphens and apostrophes
3. A username must be unique within the Company

The **default username** for a person is composed of a maximum of the first 7 characters of their **family name**, followed by as many letters of their **given name** as possible. Any **additional names** are to be disregarded. Unfortunately, that would mean Peter Parker and Penelope Parker would both have the username: **parkerpe**.

In order to deduplicate in these cases, apply the following rules in the following order each time:

1. Use more characters from the person's given name, and fewer from their family name, until a unique username is generated.
2. If the above still results in a duplicate, differentiate the username by adding a single digit at the end, starting from 1 and increasing by 1 for each duplicate.
3. If replacing a letter with a number to satisfy the length constraint, replace letters from the given name.

Always use as many characters as possible. If you have 7 characters available, do not deduplicate by using 6. Deduplicated results should never be shorter than the original username.

Function Description
Complete the function *generate_usernames* in the editor below.
generate_usernames has the following parameter(s):
usernames *Array<string>*: Names are given as individual

Language: C++11 Environment

```

1 > #include <bits/stdc++.h> ...
9
10
11 /*
12  * Complete the 'generate_usernames' function below.
13  * The function is expected to return a STRING_ARRAY.
14  * The function accepts STRING_ARRAY names as parameter.
15  */
16
17 vector<string> generate_usernames(vector<string> names) {
18
19 }
20
21 > int main() ...

```

Line: 9 Col: 1

Test Results Custom Input

Run Code Run Tests Submit

1h 5m left

2. May only contains characters in the sets [a-z] and [0-9], beware real-world names containing hyphens and apostrophes

3. A username must be unique within the Company

The **default username** for a person is composed of a maximum of the first 7 characters of their **family name**, followed by as many letters of their **given name** as possible. Any **additional names** are to be disregarded. Unfortunately, that would mean Peter Parker and Penelope Parker would both have the username: **parkerpe**.

In order to deduplicate in these cases, apply the following rules in the following order each time:

1. Use more characters from the person's given name, and fewer from their family name, until a unique username is generated.
2. If the above still results in a duplicate, differentiate the username by adding a single digit at the end, starting from 1 and increasing by 1 for each duplicate.
3. If replacing a letter with a number to satisfy the length constraint, replace letters from the given name.

Always use as many characters as possible. If you have 7 characters available, do not deduplicate by using 6. Deduplicated results should never be shorter than the original username.

Function Description
Complete the function *generate_usernames* in the editor below.
generate_usernames has the following parameter(s):
usernames *Array<string>*: Names are given as individual strings, in the following format: "<Given name> <middle or additional names> <family name>"

Returns
Array<string>: array or list of usernames satisfying the constraints above

Assumptions

- Names may contain alphabetical characters, hyphens, and apostrophes ([Aa-Zz], '-', '')
- A single digit (0-9) will always be enough to generate a unique username

Language: C++11 Environment

```

1 > #include <bits/stdc++.h> ...
9
10
11 /*
12  * Complete the 'generate_usernames' function below.
13  * The function is expected to return a STRING_ARRAY.
14  * The function accepts STRING_ARRAY names as parameter.
15  */
16
17 vector<string> generate_usernames(vector<string> names) {
18
19 }
20
21 > int main() ...

```

Line: 9 Col: 1

Test Results Custom Input

Run Code Run Tests Submit

```
#include <bits/stdc++.h>
using namespace std;
```

```
// Helper function to clean the input name (remove hyphens and apostrophes)
string clean_name(const string &name) {
    string result;
    for (char ch : name) {
        if (isalpha(ch)) {
            result += tolower(ch);
        }
    }
}
```

```

    }
    return result;
}

// Function to generate unique usernames
vector<string> generate_usernames(vector<string> names) {
    unordered_map<string, int> username_count; // To keep track of usernames used and their
counts
    vector<string> result;

    for (string &full_name : names) {
        // Split the given full name into parts (given name, middle name(s), family name)
        vector<string> name_parts;
        stringstream ss(full_name);
        string word;

        while (ss >> word) {
            name_parts.push_back(clean_name(word));
        }

        if (name_parts.size() < 2) continue; // Skip if there is not enough data to generate username

        string given_name = name_parts[0];
        string family_name = name_parts.back();

        // Start with a default username: first 7 characters of family name + as much of given name as
possible
        string username = family_name.substr(0, min(7, (int)family_name.size()));
        int remaining_length = 8 - username.length();
        if (remaining_length > 0) {
            username += given_name.substr(0, remaining_length);
        }

        // Attempt to adjust the username to make it unique by changing family/given name
composition
        int family_chars = min(7, (int)family_name.size());
        int given_chars = remaining_length;

        while (username_count.find(username) != username_count.end()) {
            if (family_chars > 1) {
                // Reduce the number of characters taken from the family name, and increase from given
name
                family_chars--;
                given_chars = 8 - family_chars;
                username = family_name.substr(0, family_chars) + given_name.substr(0, given_chars);
            } else {
                // If reducing family name characters is not enough, append a number to make it unique
                int count = ++username_count[username];
                if (username.length() == 8) {
                    // Replace the last character with a number
                    username[7] = '0' + count;
                } else {
                    // Append a number to the username
                    username += to_string(count);
                }
            }
        }

        // Store the final unique username
        username_count[username]++;
        result.push_back(username);
    }
}

```

```

    return result;
}

int main() {
    vector<string> names = {
        "Peter Parker",
        "Penelope Parker",
        "Tony Stark",
        "Bruce Banner",
        "Steve Rogers",
        "Natasha Romanoff",
        "Peter Parker", // Duplicate test
        "Penelope Parker" // Another duplicate test
    };

    vector<string> usernames = generate_usernames(names);

    for (const string &username : usernames) {
        cout << username << endl;
    }

    return 0;
}

```

QUESTION 7

<https://leetcode.com/problems/subarray-product-less-than-k/description/>

```

class Solution {
public:
    int numSubarrayProductLessThanK(vector<int>& nums, int k) {
        if (k <= 1) {
            return 0;
        }

        int i = 0, j = 0;
        int n = nums.size();
        int product = nums[0];
        int ans = 0;

        nums.push_back(1);

        while (j < n) {
            if (product < k) {
                product *= nums[++j];
            } else {
                ans += j - i;
                product /= nums[i++];
            }
        }
    }
}

```

```
    }  
  
    ans += (j - i) * (j - i + 1) / 2;  
  
    return ans;  
}  
};
```

1. Question 1

Cole volunteers as a shelver at the school library, where the encyclopedias need to be re-organized. After re-organizing all the encyclopedias, Cole realizes that some of the encyclopedias are shelved in the wrong order. To fix the mistake, Cole decides to remove all the encyclopedias from the n by m shelf. While reshelfing the encyclopedias, Cole realizes that the most efficient way to reshelve the encyclopedias is to select one encyclopedia, remove it, and remove all the other encyclopedias in the same row and column that the author wrote of the encyclopedia Cole chose. Cole wants to determine the minimum number of encyclopedias that need to be selected to remove all the encyclopedias from the shelf.

Note: The encyclopedias shelf section contains k authors. Each author is represented by an integer from 1 to k .

Example
Let there be $k = 3$ authors. Let the following matrix represent the encyclopedias shelf:

```

2 2 1
1 1 1
2 3 3

```

Cole can choose the encyclopedia at position (2,3) in the first operation and remove it. Then, the matrix looks like this:

```

2 2 x
x x x
2 3 3

```

Code Editor: Language: C++20. Autocomplete Ready.

```

1 > #include <bits/stdc++.h>
10
11 /*
12 * Complete the 'findMinimumOperations' function below.
13 *
14 * The function is expected to return an INTEGER.
15 * The function accepts following parameters:
16 * 1. 2D_INTEGER_ARRAY bookshelf
17 * 2. INTEGER k
18 */
19
20 int findMinimumOperations(vector<vector<int>> bookshelf, int k) {
21
22 }
23
24 > int main() {

```

Line: 10 Col: 1

Test Results Custom Input Run Code Run Tests Submit

Soln

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
class DisjointSet {
```

```
    vector<int> rank, parent, size;
```

```
public:
```

```
    DisjointSet(int n) {
```

```
        rank.resize(n + 1, 0);
```

```
        parent.resize(n + 1);
```

```
        size.resize(n + 1);
```

```
        for (int i = 0; i <= n; i++) {
```

```
            parent[i] = i;
```

```
            size[i] = 1;
```

```
        }
```

```
    }
```

```
    int findUPar(int node) {
```

```
        if (node == parent[node])
```

```
            return node;
```

```
        return parent[node] = findUPar(parent[node]);
```

```
    }
```

```

void unionBySize(int u, int v) {
    int ulp_u = findUPar(u);
    int ulp_v = findUPar(v);
    if (ulp_u == ulp_v) return;
    if (size[ulp_u] < size[ulp_v]) {
        parent[ulp_u] = ulp_v;
        size[ulp_v] += size[ulp_u];
    }
    else {
        parent[ulp_v] = ulp_u;
        size[ulp_u] += size[ulp_v];
    }
}
};

```

```

class Solution {
public:
    int minRemovals(vector<vector<int>>& shelf) {
        int n = shelf.size();
        int m = shelf[0].size();

        // MaxRow and MaxCol are based on matrix dimensions, we treat rows and columns separately
        DisjointSet ds(n * m);
        unordered_map<int, int> componentNodes;

        // Loop through the grid and unite nodes based on the same row and column condition
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < m; j++) {
                // Row-wise union if the same value appears in the row
                for (int k = j + 1; k < m; k++) {
                    if (shelf[i][j] == shelf[i][k]) {
                        ds.unionBySize(i * m + j, i * m + k); // Connect row cells if values are the same
                    }
                }
            }

            // Column-wise union if the same value appears in the column
            for (int k = i + 1; k < n; k++) {
                if (shelf[i][j] == shelf[k][j]) {
                    ds.unionBySize(i * m + j, k * m + j); // Connect column cells if values are the same
                }
            }

            // Mark the current cell as seen
            componentNodes[i * m + j] = 1;
        }
    }
};

```

```

    }

    // Count unique components
    int cnt = 0;
    for (auto it : componentNodes) {
        if (ds.findUPar(it.first) == it.first) {
            cnt++;
        }
    }

    // The number of removals is the total number of components
    return cnt;
}

};

int main() {
    vector<vector<int>>> shelf = {
        {2, 2, 1},
        {2, 1, 1},
        {3, 3, 3}
    };

    Solution sol;
    cout << "Minimum removals to clean the shelf: " << sol.minRemovals(shelf) << endl;

    return 0;
}

```

```

1 1 2
2 2 2
1 2 2

```

output = 3

Alt Ans.

```

def find_idx(L):
    num=max(L)
    for j in range (0,len(L)):
        if L[j]==num:
            return j

def get(L):
    ans=0
    visited=[False]*len(L)

```



```

while False in visited:
    x=[0]*n
    y=[0]*m
    for j in range (0,len(L)):
        if visited[j]==True:
            continue
        u,v=L[j]
        x[u]=x[u]+1
        y[v]=y[v]+1
    u=find_idx(x)
    v=find_idx(y)
    for j in range (0,len(L)):
        if L[j][0]==u or L[j][1]==v:
            visited[j]=True
    ans=ans+1
return ans

```

```

for j in range (0,len(L)):
    for k in range (0,len(L)):
        M=dic.get(L[j][k],[])
        M.append([j,k])
        dic[L[j][k]]=M
ans=0
for x in dic:
    count=get(dic[x])
    ans=ans+count
print(ans)

```

C++

```

#include <iostream>
#include <vector>
#include <unordered_map>
#include <algorithm>

using namespace std;

int find_idx(const vector<int> &L) {
    int num = *max_element(L.begin(), L.end());
    for (int j = 0; j < L.size(); ++j) {
        if (L[j] == num) {
            return j;
        }
    }
}

```

```

    }
}
return -1; // This line should never be reached if the list is non-empty.
}

```

```

int get(const vector<pair<int, int>> &L, int n, int m) {
    int ans = 0;
    vector<bool> visited(L.size(), false);

    while (find(visited.begin(), visited.end(), false) != visited.end()) {
        vector<int> x(n, 0), y(m, 0);

        for (int j = 0; j < L.size(); ++j) {
            if (visited[j]) {
                continue;
            }

            int u = L[j].first;
            int v = L[j].second;
            x[u]++;
            y[v]++;
        }

        int u = find_idx(x);
        int v = find_idx(y);

        for (int j = 0; j < L.size(); ++j) {
            if (L[j].first == u || L[j].second == v) {
                visited[j] = true;
            }
        }

        ans++;
    }

    return ans;
}

```

```

int main() {
    int n = 3, m = 3, k = 2;
    vector<vector<int>> L = {
        {1, 1, 2},
        {2, 2, 2},
        {1, 2, 2}
    };
}

```

```

unordered_map<int, vector<pair<int, int>>> dic;

// Create the dictionary to group indices by value
for (int j = 0; j < L.size(); ++j) {
    for (int k = 0; k < L[j].size(); ++k) {
        dic[L[j]][k].emplace_back(j, k);
    }
}

// Calculate the answer
int ans = 0;
for (auto &entry : dic) {
    int count = get(entry.second, n, m);
    ans += count;
}

cout << ans << endl;

return 0;
}

```

Hackerank Approach

For each unique value from 1 to K, greedily find the best intersection point, destroy the cells, keep doing the same until all cells with that value has been destroyed. To find the best intersection point it takes $O(n*m)$, and the number of steps to destroy all would be at most (n,m) for each unique value, and there are at most K unique values. Therefore, the overall time complexity would be $O(k*\min(n,m)*n*m)$. Considering the worst case where $n=m=100$, $k=50$, it turns out to be $O(50*100*100*100) = O(5*10^7)$.

QUESTION 2:

2. Question 2

Process an array, *arr*, using two pointers, *P1* and *P2*, and an integer, *segSize*. Pick an integer to test for *segSize*, then start with *P1* = 0 and *P2* = 1. Compare the value at *arr[P1]* with all of the elements in a subarray. The subarray starts at *arr[P2]* and includes *segSize* elements or it reaches the end of *arr*. If *arr[P1]* is greater or equal in all cases, increment *P1* by 1, increment *P2* by *segSize*, and repeat the process. Do this until the subarray including the last element of *arr* has been processed. Determine the minimum value of *segSize* that allows the entire array to be processed. Return this minimum step value or -1 if it is not possible.

Example:
arr = [11, 9, 10, 8, 10, 9]

Try: *segSize* = 1

P1	P2	arr[P1]	Subarray	Continue
0	1	11	[9]	y
1	2	9	[10]	n (fails because 10 > 9)

Try: *segSize* = 2

P1	P2	arr[P1]	Subarray	Continue
0	1	11	[9, 10]	y
1	3	9	[8, 10]	n (fails because 10 > 9)

Try: *segSize* = 3

P1	P2	arr[P1]	Subarray	Continue
0	1	11	[9, 10, 8]	y
1	4	9	[10, 9]	n (fails because 10 > 9)

Try: *segSize* = 4

P1	P2	arr[P1]	Subarray	Continue
0	1	11	[9, 10, 8, 10]	y
1	5	9	[9]	y

All array elements were processed, so the minimum *segSize* = 4.

Function Description

0 1 11 19, 10, 8, 10
1 5 9 [9] y

All array elements were processed, so the minimum *segSize* = 4.

Function Description

Complete the function *dualSpeed* in the editor below. The function must return a single integer, which is the answer to the problem.

dualSpeed has the following parameter:

- arr*[*arr*[0]...*arr*[*n*-1]]: an array of integers

Constraints

- $1 \leq n \leq 5000$
- $1 \leq arr[i] \leq 10^9$

Input Format For Custom Testing

Sample Case 0

Sample Input For Custom Testing

STDIN	Function
7	+ <i>arr</i> [] size integer <i>n</i> = 7
11	+ <i>arr</i> = [11, 9, 7, 7, 7, 6, 5]
9	
7	
7	
6	
5	

Sample Output

1

Explanation

arr = [11, 9, 7, 7, 7, 6, 5]

Since the array is in non-increasing order, a *segSize* of 1 can be used.

Sample Case 1

1m 16s left

2. Question 2

Process an array, *arr*, using two pointers, *P1* and *P2*, and an integer, *segSize*. Pick an integer to test for *segSize*, then start with *P1* = 0 and *P2* = 1. Compare the value at *arr[P1]* with all of the elements in a subarray. The subarray starts at *arr[P2]* and includes *segSize* elements or it reaches the end of *arr*. If *arr[P1]* is greater or equal in all cases, increment *P1* by 1, increment *P2* by *segSize*, and repeat the process. Do this until the subarray including the last element of *arr* has been processed. Determine the minimum value of *segSize* that allows the entire array to be processed. Return this minimum step value or -1 if it is not possible.

Example:
arr = [11, 9, 10, 8, 10, 9]

Try: <i>segSize</i> = 1	P1	P2	arr[P1]	Subarray	Continue
	0	1	11	[9]	y
	1	2	9	[10]	n (fails because 10 > 9)

Try: <i>segSize</i> = 2	P1	P2	arr[P1]	Subarray	Continue
	0	1	11	[9, 10]	y
	1	3	9	[8, 10]	n (fails because 10 > 9)

Try: <i>segSize</i> = 3	P1	P2	arr[P1]	Subarray	Continue
	0	1	11	[9, 10, 8]	y
	1	4	9	[10, 9]	n (fails because 10 > 9)

Try: <i>segSize</i> = 4	P1	P2	arr[P1]	Subarray	Continue
	0	1	11	[9, 10, 8, 10]	y
	1	5	9	[9]	y

All array elements were processed, so the minimum *segSize* = 4.

Function Description

Complete the function *dualSpeed* in the editor below. The function must return a single integer,

Language C++20
Environment
Autocomplete Ready

```

1 #include <algorithm>
2 #include <bits/stdc++.h>
3
4 using namespace std;
5
6 string ltrim(const string &);
7 string rtrim(const string &);
8
9
10 /*
11  * Complete the 'dualSpeed' function below.
12  *
13  * The function is expected to return an INTEGER.
14  * The function accepts INTEGER_ARRAY arr as parameter.
15  */
16 bool check(int mid,int a,int b,vector<int> &arr){
17     //bool flag1=false;
18     // a=0,b=1;
19     int n=arr.size();
20     int last=min(n,b+mid);
21     int max1=*max_element(arr.begin()+b,arr.begin()+last-1)
22
23     while(arr[a]>=max1){
24         //cout<<arr[a]<<" "<<max1<<" "<<last<<endl;
25         a++;
26         b+=mid;
27         last=min(n,b+mid);
28         if(b>=n-1){return true;}
29         max1=*max_element(arr.begin()+b,arr.begin()+last-1)
30     }
31     return b>=n-1;

```

Test Results
Custom Input
Run Code
Run Tests
Submit

started with segsize = 1 , checked condition at each step until the subarray having last element
if condition met return segsize , else continue to segsize + 1 and repeat the process.

```

#include<bits/stdc++.h>
bool canProcessWithSegSize(const std::vector<int>& arr, int segSize) {
    int n = arr.size();
    int P1 = 0;
    int P2 = 1;
    while (P1 < n && P2 < n)
    {
        bool isValid = true;
        for (int i = 0; i < segSize && P2 + i < n; ++i)
        {
            if (arr[P1] < arr[P2 + i])
            {
                isValid = false;
                break;
            }
        }
        if (isValid)

```

```

        {
            P1++;
            P2 += segSize;
        } else
        {
            return false;
        }
    }

    return true;
}

int findMinimumSegSize(const std::vector<int>& arr) {
    int n = arr.size();

    if (n <= 1) {
        return 1;
    }

    for (int segSize = 1; segSize < n; ++segSize) {
        if (canProcessWithSegSize(arr, segSize)) {
            return segSize;
        }
    }

    return -1;
}

int main() {
    std::vector<int> arr = {11,9,10,8,10,9}; // Example array

    int result = findMinimumSegSize(arr);

    if (result != -1) {
        std::cout << "Minimum segSize: " << result << std::endl;
    } else {
        std::cout << "No valid segSize found." << std::endl;
    }

    return 0;
}

```

7. Linux: Users Bulk Onboarding

- Read the list of first and last names of employers to onboard from `"/home/ubuntu/900026-linux-users-bulk-onboarding/onboard.txt"`.

- Note
- The completed solution will be evaluated in a new, clean environment. Be sure everything is in the `"/home/ubuntu/900026-linux-users-bulk- onboarding"` folder.
- All the tasks should be done within a simple `"sudo solve"` execution invoked from the question directory.

Note

The completed solution will be evaluated in a new, clean environment. Be sure everything is in the `"/home/ubuntu/900026-linux-users-bulk- onboarding"` folder.

All the tasks should be done within a simple "sudo solve" execution invoked from the question directory.

- You have sudo access.

Grading

- The execution result of "sudo solve" invoked from the question directory solves the task.

Solution

```
#!/bin/bash
```

```
# Define the file path
```

```
FILE="/home/ubuntu/900026-linux-users-bulk-onboarding/onboard.txt"
```

```
# Check if the file exists
```

```
# Check if the file exists
if [ ! -f "$FILE" ]; then
    echo "File not found!"
    exit 1
fi
```

fi

```
# Create the "onboarding" group if it doesn't exist
```

```
if ! getent group onboarding >/dev/null; then
```

```
sudo groupadd onboarding
```

fi

```
# Loop through the onboard.txt file
```

```
while IFS=' ' read -r firstname lastname
```

do

```
# Format the username as f_lastname (lowercase first letter of the first name)
```

```
username="${firstname:0:1,,} $lastname"
```

```
# Create the user with the formatted username
```

```
if id "$username" &>/dev/null; then
```

```
echo "User $username already exists, skipping..."
```

else

```
sudo useradd -m -g onboarding "$username"
```

```
echo "User $username created."
```

fi

```
# Force user to change password at next login
```

```
sudo passwd -e "$username"
```

```
done < "$FILE"
```

```
echo "All users onboarded successfully."
```

Running the command

```
sudo chmod +x /home/ubuntu/900026-linux-users-bulk-onboarding/script.sh
```

```
sudo /home/ubuntu/900026-linux-users-bulk-onboarding/script.sh
```

DOCKER:

The screenshot shows a task interface on the left and a terminal window on the right. The task interface has a sidebar with a progress bar showing steps 1 through 9. Step 8, '8. Docker: Port Binding', is the current step. The task description states: 'The task is to complete a file stub "/home/ubuntu/826087-docker-port-binding/script.sh" with one or more steps that do the following:'. It lists two bullet points: 'Runs a new Docker container "my-container" from the "nginx" image (latest tag) in interactive background mode, without pseudo-TTY allocation.' and 'Binds port "80" of "my-container" container to port "8000" on localhost.'. Below this is a 'Note' section with four bullet points: 'The completed solution will be evaluated in a new, clean environment. Be sure everything is in the "/home/ubuntu/826087-docker-port-binding" folder.', 'All the tasks should be done within a simple "sudo solve" execution invoked from the question directory.', 'Docker is already installed.', and 'You have sudo access.'. The terminal window on the right shows the command 'bash: /home/ubuntu/826087-docker-port-binding/script.sh: Permission denied' and a 'Submit' button at the bottom right.

Step 1: Create the script directory

First, ensure that the target directory exists. If it does not, create it:

```
bash
Copy code
mkdir -p /home/ubuntu/826087-docker-port-binding
```

Step 2: Create or edit the script file

You can use a text editor like nano to create the script:

```
bash
Copy code
nano /home/ubuntu/826087-docker-port-binding/script.sh
```

Step 3: Add the necessary commands to the script

In the nano editor, add the following lines to the script:

```
bash
Copy code
#!/bin/bash
# Run a new Docker container from the nginx image
sudo docker run -d --name my-container -p 8000:80 nginx
```

Step 4: Save and exit the editor

- Press CTRL + X to exit.
- Press Y to confirm saving changes.
- Press ENTER to save with the same filename.

Step 5: Ensure the script has execute permissions

Run the following command to make the script executable:

```
bash
Copy code
chmod +x /home/ubuntu/826087-docker-port-binding/script.sh
```

Step 6: Run the script

Change to the script's directory and execute it:

```
bash
Copy code
cd /home/ubuntu/826087-docker-port-binding
sudo ./script.sh
```

Step 7: Verify that the container is running

You can check that the Nginx welcome page is accessible by executing:

```
bash
Copy code
wget -qO- http://localhost:8000
```

Expected Output

If everything is set up correctly, you should see the HTML content of the Nginx welcome page as the output of the wget command.

Summary of Commands

Here's a summary of all the commands you need to run in order:

```
bash
Copy code
mkdir -p /home/ubuntu/826087-docker-port-binding
nano /home/ubuntu/826087-docker-port-binding/script.sh
# (Add the script content and save)
```

```
chmod +x /home/ubuntu/826087-docker-port-binding/script.sh
cd /home/ubuntu/826087-docker-port-binding
sudo ./script.sh
wget -qO- http://localhost:8000
```

This will create the required Docker container and bind the specified ports.

REST API

1h 9m left

9. REST API: City Weather Station

ALL

Use the *HTTP GET* method to retrieve information from a database of weather records. Query `https://jsonmock.hackerrank.com/api/weather/search?name={keyword}` to search all the records where `name` has a keyword anywhere in its string value. The query result is paginated and can be further accessed by appending to the query string `&page=num` where `num` is the page number.

The query response from the API is a JSON with the following five fields:

- `page`: the current page
- `per_page`: the maximum number of results per page
- `total`: the total number of records in the search result
- `total_pages`: the total number of pages to query to get all the results
- `data`: an array of JSON objects that contain weather records

The data field in the response contains a list of weather records with the following schema:

- `name`: the name of the city
- `weather`: temperature recorded
- `status`: an array of wind speed and humidity records

Filter records to include a given keyword in the `name` parameter. Return an array such that each

Language C++11 Environment Autocomplete Ready

```
1 > #include <bits/stdc++.h> ...
11 /*
12  * Complete the 'weatherStation' function below.
13  *
14  * The function is expected to return a STRING_ARRAY.
15  * The function accepts STRING keyword as parameter.
16  *
17  * Base URL: https://jsonmock.hackerrank.com/api/weather/search?name={keyword}
18  *
19  */
20
21 vector<string> weatherStation(string keyword) {
22
23 }
24 > int main() ...
```

Line: 11 Col: 1

Test Results Custom Input

Run Code Run Tests Submit

1h 9m left

Filter records to include a given keyword in the name parameter. Return an array such that each element is a string of comma-separated values: *city_name, temperature, wind, humidity*.

For example,

```
{
  "name": "Adelaide",
  "weather": "15 degree",
  "status": {
    "Wind: 8Kmph",
    "Humidity: 61%"
  }
}
```

The JSON record is stored as *Adelaide,15,8,61*
Return the list sorted by city name.

Function Description
Complete the function *weatherStation(keyword)* in the editor below.

weatherStation has the following parameter(s):
string keyword: the string to search

Returns
string[]: the weather data for each city

Note: Please review the header in the code stub to see available libraries for API requests in the selected language. Required libraries can be imported in order to solve the question. Check our full list of supported libraries at <https://www.hackerrank.com/environment>.

► Input Format For Custom Testing

▼ Sample Case 0

Sample Input For Custom Testing

Language C++11 Environment Autocomplete Ready

```
1 > #include <bits/stdc++.h> ...
11 /*
12  * Complete the 'weatherStation' function below.
13  *
14  * The function is expected to return a STRING_ARRAY.
15  * The function accepts STRING keyword as parameter.
16  *
17  * Base URL: https://jsonmock.hackerrank.com/api/weather/search?name={keyword}
18  *
19  */
20
21 vector<string> weatherStation(string keyword) {
22
23 }
24 > int main() ...
```

Line: 11 Col: 1

Test Results Custom Input

Run Code Run Tests Submit

1h 9m left

Returns
string[]: the weather data for each city

Note: Please review the header in the code stub to see available libraries for API requests in the selected language. Required libraries can be imported in order to solve the question. Check our full list of supported libraries at <https://www.hackerrank.com/environment>.

► Input Format For Custom Testing

▼ Sample Case 0

Sample Input For Custom Testing

all

Sample Output

Dallas,12,2,5
Dallupura,10,9,14
Vallejo,1,24,56

Explanation

The list of cities that contain the keyword 'all' and their data is shown as comma-separated values.

▼ Sample Case 1

Sample Input For Custom Testing

io

Sample Output

Qiongsan,3,31,47

Explanation

Only one city contains the keyword 'io'.

Language C++11 Environment Autocomplete Ready

```
1 > #include <bits/stdc++.h> ...
11 /*
12  * Complete the 'weatherStation' function below.
13  *
14  * The function is expected to return a STRING_ARRAY.
15  * The function accepts STRING keyword as parameter.
16  *
17  * Base URL: https://jsonmock.hackerrank.com/api/weather/search?name={keyword}
18  *
19  */
20
21 vector<string> weatherStation(string keyword) {
22
23 }
24 > int main() ...
```

Line: 11 Col: 1

Test Results Custom Input

Run Code Run Tests Submit

hackerank.com/test/34hgrraoia8/questions/23dpaeita3

9. REST API: City Weather Station

Use the HTTP GET method to retrieve information from a database of weather records. Query [https://jsonmock.hackerrank.com/api/weather/search?name=\(keyword\)](https://jsonmock.hackerrank.com/api/weather/search?name=(keyword)) to search all the records where name has a keyword anywhere in its string value. The query result is paginated and can be further accessed by appending to the query string `&page=num` where `num` is the page number.

The query response from the API is a JSON with the following five fields:

- `page`: the current page
- `per_page`: the maximum number of results per page
- `total`: the total number of records in the search result
- `total_pages`: the total number of pages to query to get all the results
- `data`: an array of JSON objects that contain weather records

The data field in the response contains a list of weather records with the following schema:

- `name`: the name of the city
- `weather`: temperature recorded
- `status`: an array of wind speed and humidity records

Filter records to include a given keyword in the name parameter. Return an array such that each element is a string of comma-separated values: `city_name, temperature, wind, humidity`.

For example,

```

1 #include <bits/stdc++.h>
2 #include <jsoncpp/json/json.h>
3 #include <jsoncpp/json/reader.h>
4 #include <jsoncpp/json/writer.h>
5 #include <jsoncpp/json/value.h>
6 #include <curl/curl.h>
7
8 using namespace std;
9
10
11
12 /*
13  * Complete the 'weatherStation' function below.
14  *
15  * The function is expected to return a STRING_ARRAY.
16  * The function accepts STRING keyword as parameter.
17  *
18  * Base URL: https://jsonmock.hackerrank.com/api/weather/search?name=(keyword)
19  */
20
21
22 vector<string> weatherStation(string keyword) {
23
24 }
25
26 int main()
27 {
28     ofstream fout(getenv("OUTPUT_PATH"));
29
30     string keyword;
31     getline(cin, keyword);
32
33     vector<string> result = weatherStation(keyword);
34

```

Test Results Custom Input Run Code Run Tests Submit

hackerank.com/test/34hgrraoia8/questions/23dpaeita3

For example,

```

{
  "name": "Adelaide",
  "weather": "15 degree",
  "status": {
    "Wind": 8Kmph",
    "Humidity": 61%"
  }
}

```

The JSON record is stored as `Adelaide,15.8,61`
Return the list sorted by city name.

Function Description
Complete the function `weatherStation(keyword)` in the editor below.

`weatherStation` has the following parameter(s):
string keyword: the string to search

Returns
string[]: the weather data for each city

Note: Please review the header in the code stub to see available libraries for API requests in the selected language. Required libraries can be imported in order to solve the question. Check our full list of supported libraries at <https://www.hackerrank.com/environment>.

▶ Input Format For Custom Testing

▼ Sample Case 0

Sample Input For Custom Testing

```
all
```

Sample Output

```
Dallas,12,2,5
Dellupura,18,9,14
Vallejo,1,24,56
```

Explanation

The list of cities that contain the keyword 'all' and their data is shown.

```

1 #include <bits/stdc++.h>
2 #include <jsoncpp/json/json.h>
3 #include <jsoncpp/json/reader.h>
4 #include <jsoncpp/json/writer.h>
5 #include <jsoncpp/json/value.h>
6 #include <curl/curl.h>
7
8 using namespace std;
9
10
11
12 /*
13  * Complete the 'weatherStation' function below.
14  *
15  * The function is expected to return a STRING_ARRAY.
16  * The function accepts STRING keyword as parameter.
17  *
18  * Base URL: https://jsonmock.hackerrank.com/api/weather/search?name=(keyword)
19  */
20
21
22 vector<string> weatherStation(string keyword) {
23
24 }
25
26 int main()
27 {
28     ofstream fout(getenv("OUTPUT_PATH"));
29
30     string keyword;
31     getline(cin, keyword);
32
33     vector<string> result = weatherStation(keyword);
34
35     for (size_t i = 0; i < result.size(); i++) {
36         fout << result[i];
37
38         if (i != result.size() - 1) {

```

Test Results Custom Input Run Code Run Tests Submit

```

#include <iostream>
#include <string>
#include <vector>
#include <sstream>
#include <algorithm>
#include <curl/curl.h>
#include <json/json.h> // Make sure you have a JSON library installed

```

using namespace std;

```

// Callback function to handle the response data from cURL
size_t WriteCallback(void* contents, size_t size, size_t nmemb, string* userp) {
    size_t totalSize = size * nmemb;
    userp->append((char*)contents, totalSize);
    return totalSize;
}

```

```

}

vector<string> weatherStation(string keyword) {
    vector<string> results;
    CURL* curl;
    CURLcode res;
    string readBuffer;

    // Initialize cURL
    curl_global_init(CURL_GLOBAL_DEFAULT);
    curl = curl_easy_init();

    if (curl) {
        // Set the URL for the API request
        string url = "https://jsonmock.hackerrank.com/api/weather/search?name=" + keyword;
        curl_easy_setopt(curl, CURLOPT_URL, url.c_str());
        curl_easy_setopt(curl, CURLOPT_WRITEFUNCTION, WriteCallback);
        curl_easy_setopt(curl, CURLOPT_WRITEDATA, &readBuffer);

        // Perform the request
        res = curl_easy_perform(curl);

        // Check for errors
        if (res != CURLE_OK) {
            cerr << "curl_easy_perform() failed: " << curl_easy_strerror(res) << endl;
            return results;
        }

        // Clean up
        curl_easy_cleanup(curl);
    }

    curl_global_cleanup();

    // Parse the JSON response
    Json::Value jsonData;
    Json::Reader jsonReader;

    if (jsonReader.parse(readBuffer, jsonData)) {
        // Process the 'data' array from the JSON response
        const Json::Value data = jsonData["data"];

        // Iterate through each record
        for (const auto& record : data) {
            string cityName = record["name"].asString();
            string temperature = record["weather"].asString();
            const Json::Value status = record["status"];

            // Assuming status contains "Wind: XKmph" and "Humidity: Y%"
            string wind = status[0].asString(); // Get wind status
            string humidity = status[1].asString(); // Get humidity status

            // Extract numerical values for wind and humidity
            size_t windStart = wind.find(':') + 2; // Find wind speed after "Wind: "
            size_t windEnd = wind.find('K', windStart); // Find 'K'
            string windSpeed = wind.substr(windStart, windEnd - windStart); // Extract wind speed

            size_t humidityStart = humidity.find(':') + 2; // Find humidity after "Humidity: "
            size_t humidityEnd = humidity.find('%', humidityStart); // Find '%'
            string humidityValue = humidity.substr(humidityStart, humidityEnd - humidityStart); // Extract
            humidity

            // Create a result string in the desired format
            results.push_back(cityName + ", " + temperature + ", " + windSpeed + ", " + humidityValue);
        }
    } else {
        cerr << "Failed to parse JSON data" << endl;
    }

    // Sort results by city name
    sort(results.begin(), results.end());

    return results;
}

int main() {
    string keyword;
    cout << "Enter the keyword: ";
    cin >> keyword;

    vector<string> weatherData = weatherStation(keyword);
    for (const auto& record : weatherData) {
        cout << record << endl;
    }

    return 0;
}

```

MCQ

30 September 2024

12:06